DAX

CHEAT SHEET

Data Analysis Expressions — Power BI / Analysis Services

How to Read This Sheet

Every function table includes a colour-coded badge showing where each function can be used:

COLUMN ONLY	MEASURE ONLY	COLUMN & MEASURE

Context	What it means
`Column`	Calculated column — evaluated row by row in row context. Result stored in the model.
`Measure`	Evaluated dynamically at query time in filter context. Not stored — computed on demand.
`Both`	Can be used in either context. Behaviour may differ slightly depending on where used.

Core Concepts

Concept	Description
`Calculated Column`	Computed row by row in row context. Stored in the model. Use for per-row values like categories, flags, or concatenated labels.
`Measure`	Computed dynamically at query time based on filter context. Use for aggregations like totals, ratios, and KPIs.
`Row Context`	Exists inside calculated columns and iterator (X) functions. References the current row being evaluated.
`Filter Context`	Exists in measures. Defined by slicers, rows/columns in visuals, and CALCULATE filter arguments.
`Context Transition`	When a measure is called inside a calculated column or CALCULATE, row context is converted to filter context.

Aggregation Functions

Function	Description	Use
`SUM(column)`	Sum of a column	Measure
`AVERAGE(column)`	Average of a column	Measure
`MIN(column)`	Minimum value	Both
`MAX(column)`	Maximum value	Both
`COUNT(column)`	Count non-blank values	Measure
`COUNTA(column)`	Count non-blank values (any type)	Measure
`COUNTROWS(table)`	Count rows in a table	Measure
`DISTINCTCOUNT(column)`	Count of unique values	Measure
`DISTINCTCOUNTNOBLANK(column)`	Unique values excluding blanks	Measure

Note: Aggregation functions like SUM and COUNT require filter context to be meaningful — they are almost always used as measures.

Iterator (X) Functions

Iterators loop row by row over a table, evaluate an expression per row, then aggregate the results. They always work in row context internally.

Function	Description	Use
`SUMX(table, expr)`	Sum of expression per row	Measure
`AVERAGEX(table, expr)`	Average of expression per row	Measure
`COUNTX(table, expr)`	Count non-blank results per row	Measure
`MINX(table, expr)`	Min of expression per row	Measure
`MAXX(table, expr)`	Max of expression per row	Measure
`RANKX(table, expr)`	Rank value within a table	Measure
`PRODUCTX(table, expr)`	Multiply expression results per row	Measure
`CONCATENATEX(table, expr, delim)`	Concatenate expression results per row	Both

```
Revenue = SUMX(Sales, Sales[Quantity] * Sales[Unit Price])
Top Product = MAXX(Products, [Total Sales])
```

CALCULATE — The Most Important Function

CALCULATE evaluates an expression in a modified filter context. It is the engine behind almost every advanced DAX measure.

```
CALCULATE(expression, filter1, filter2, ...)
```

Usage	Example
`Force a filter`	CALCULATE([Total Sales], Geography[Country] = "UK")
`Remove all filters`	CALCULATE([Total Sales], ALL(Sales))
`Time intelligence`	CALCULATE([Total Sales], SAMEPERIODLASTYEAR(Date[Date]))
`Multiple filters`	CALCULATE([Total Sales], Region[Region]="North", Category[Cat]="Electronics")

CALCULATE is Measure only. It requires filter context to operate — it cannot be used in a calculated column without EARLIER or context transition.

CALCULATE = MEASURE ONLY

Filter Functions

Function	Description	Use
`ALL(table or column)`	Remove all filters from table or column	Measure
`ALLEXCEPT(table, col1, col2)`	Remove all filters except specified columns	Measure
`ALLSELECTED(table or column)`	Keep only slicer/visual filters	Measure
`FILTER(table, condition)`	Return a filtered table	Both
`KEEPFILTERS(expression)`	Keep existing filters, do not override	Measure
`REMOVEFILTERS(table or col)`	Remove filters — alias for ALL in CALCULATE	Measure
`SELECTEDVALUE(col, alt)`	Return value if single item selected	Measure
`HASONEVALUE(column)`	TRUE if column has a single value in context	Measure
`HASONEFILTER(column)`	TRUE if column has exactly one direct filter	Measure

`ISFILTERED(column)`	TRUE if column has any direct filter applied	**Measure**
`ISCROSSFILTERED(column)`	TRUE if column is filtered via a relationship	**Measure**
`VALUES(column)`	Distinct values as a table, including blank	**Measure**
`DISTINCT(column)`	Distinct values excluding blank	**Both**

Time Intelligence Functions

Requires a continuous date table with no gaps, marked as a Date Table in the model.

Function	Description	Use
`TOTALYTD(expr, dates)`	Year-to-date total	Measure
`TOTALQTD(expr, dates)`	Quarter-to-date total	Measure
`TOTALMTD(expr, dates)`	Month-to-date total	Measure
`SAMEPERIODLASTYEAR(dates)`	Same period in the prior year	Measure
`DATEADD(dates, n, interval)`	Shift dates by n intervals (DAY/MONTH/QUARTER/YEAR)	Measure
`DATESMTD(dates)`	Dates from month start to current date	Measure
`DATESQTD(dates)`	Dates from quarter start to current date	Measure
`DATESYTD(dates)`	Dates from year start to current date	Measure
`PREVIOUSMONTH(dates)`	Full previous month	Measure
`PREVIOUSQUARTER(dates)`	Full previous quarter	Measure
`PREVIOUSYEAR(dates)`	Full previous year	Measure
`NEXTMONTH(dates)`	Full next month	Measure
`PARALLELPERIOD(dates, n, interval)`	Shift period in parallel	Measure
`DATESINPERIOD(dates, start, n, interval)`	Dates in a custom rolling period	Measure
`FIRSTDATE(dates)`	First date in current context	Measure
`LASTDATE(dates)`	Last date in current context	Measure

```
YTD Sales = TOTALYTD([Total Sales], Date[Date])
LY Sales  = CALCULATE([Total Sales], SAMEPERIODLASTYEAR(Date[Date]))
MoM Chg % = DIVIDE([Total Sales] - [LM Sales], [LM Sales])
```

Logical Functions

Function	Description	Use
`IF(cond, true, false)`	Conditional — return one of two values	Both
`IFERROR(expr, fallback)`	Return fallback if expression errors	Both
`SWITCH(expr, v1, r1, ..., else)`	Multi-condition switch	Both
`AND(a, b)`	Logical AND — returns TRUE/FALSE	Both
`OR(a, b)`	Logical OR — returns TRUE/FALSE	Both
`NOT(condition)`	Negate a boolean	Both
`BLANK()`	Return a blank value	Both
`ISBLANK(expr)`	Check if a value is blank	Both
`ISNUMBER(expr)`	Check if a value is a number	Both
`ISTEXT(expr)`	Check if a value is text	Both
`ISLOGICAL(expr)`	Check if a value is boolean	Both
`ISERROR(expr)`	Check if expression returns an error	Both

```
Category =
SWITCH(TRUE(),
    [Sales] > 10000, "High",
    [Sales] > 5000,  "Medium",
    "Low"
)
```

Text Functions

Function	Description	Use
`CONCATENATE(a, b)`	Join two strings (use & for more)	Both
`COMBINEVALUES(delim, a, b, ...)`	Join multiple strings with delimiter	Column
`LEFT(text, n)`	First n characters	Both
`RIGHT(text, n)`	Last n characters	Both
`MID(text, start, n)`	Extract from middle	Both
`LEN(text)`	String length	Both
`UPPER(text)`	Convert to uppercase	Both
`LOWER(text)`	Convert to lowercase	Both
`TRIM(text)`	Remove extra spaces	Both
`SUBSTITUTE(text, old, new)`	Replace a substring	Both
`REPLACE(text, start, n, new)`	Replace by position	Both
`FIND(find, text)`	Position of substring (case sensitive)	Both
`SEARCH(find, text)`	Position of substring (case insensitive)	Both
`FORMAT(value, format_str)`	Format a value as a string	Both
`VALUE(text)`	Convert text to number	Both
`FIXED(num, decimals)`	Format number as text with fixed decimals	Both
`REPT(text, n)`	Repeat a string n times	Both

Date & Time Functions

Function	Description	Use
`TODAY()`	Current date	Both
`NOW()`	Current date and time	Both
`DATE(year, month, day)`	Create a date from parts	Both
`TIME(hour, min, sec)`	Create a time value	Both
`YEAR(date)`	Extract year	Both
`MONTH(date)`	Extract month number	Both
`DAY(date)`	Extract day number	Both
`HOUR(datetime)`	Extract hour	Both
`MINUTE(datetime)`	Extract minute	Both
`WEEKDAY(date, mode)`	Day of week number	Both
`WEEKNUM(date)`	Week number	Both
`EOMONTH(date, months)`	Last day of the month	Both
`DATEDIFF(start, end, interval)`	Difference between two dates	Both
`EDATE(date, months)`	Date n months forward or back	Both
`CALENDARAUTO()`	Auto-generate a date table	Measure
`CALENDAR(start, end)`	Generate a date table from a range	Measure

Math & Statistical Functions

Function	Description	Use
`DIVIDE(num, denom, alt)`	Safe division — returns alt on divide by zero	Both
`ROUND(num, decimals)`	Round to n decimal places	Both
`ROUNDUP(num, decimals)`	Round up	Both
`ROUNDDOWN(num, decimals)`	Round down	Both
`INT(num)`	Round down to integer	Both
`ABS(num)`	Absolute value	Both
`SQRT(num)`	Square root	Both
`POWER(num, exp)`	Raise to a power	Both
`MOD(num, divisor)`	Remainder after division	Both
`TRUNC(num, decimals)`	Truncate to n decimal places	Both
`CEILING(num, significance)`	Round up to nearest significance	Both
`FLOOR(num, significance)`	Round down to nearest significance	Both
`MEDIAN(column)`	Median value	Measure
`PERCENTILE.INC(column, k)`	k-th percentile (inclusive)	Measure
`STDEV.S(column)`	Standard deviation (sample)	Measure
`STDEV.P(column)`	Standard deviation (population)	Measure
`VAR.S(column)`	Variance (sample)	Measure
`GEOMEAN(column)`	Geometric mean	Measure

Table Functions

Function	Description	Use
`FILTER(table, condition)`	Return filtered rows as a table	Both
`ALL(table)`	All rows ignoring filters	Measure
`VALUES(column)`	Unique values as a one-column table	Measure
`DISTINCT(column)`	Distinct values, excludes blank	Both
`ADDCOLUMNS(table, name, expr)`	Add calculated columns to a table	Both
`SELECTCOLUMNS(table, name, expr)`	Select and rename columns	Both
`SUMMARIZE(table, groupby, name, expr)`	Group and aggregate	Both
`SUMMARIZECOLUMNS(...)`	Modern version of SUMMARIZE	Measure
`CROSSJOIN(table1, table2)`	Cartesian product of two tables	Both
`UNION(table1, table2)`	Stack tables vertically	Both
`INTERSECT(table1, table2)`	Rows in both tables	Both
`EXCEPT(table1, table2)`	Rows in table1 not in table2	Both
`TOPN(n, table, expr)`	Top N rows by expression	Both
`SAMPLE(n, table, expr)`	Random N rows	Both
`ROW(name, expr)`	Create a single-row table	Measure

`DATATABLE(col, type, data)`	Create a static table inline	**Both**
`GENERATESERIES(start, end, step)`	Generate a number series	**Both**
`NATURALINNERJOIN(t1, t2)`	Inner join on common columns	**Both**
`NATURALLEFTOUTERJOIN(t1, t2)`	Left join on common columns	**Both**

Relationship Functions

Function	Description	Use
`RELATED(column)`	Get value from related table (many to one side)	Column
`RELATEDTABLE(table)`	Get related rows as a table (one to many side)	Both
`USERELATIONSHIP(col1, col2)`	Activate an inactive relationship in CALCULATE	Measure
`CROSSFILTER(col1, col2, dir)`	Change cross-filter direction	Measure
`TREATAS(table, col1, col2)`	Apply a table as filter on specified columns	Measure

RELATED is Column only — it uses row context to look up a value from the related (one) side of a relationship. RELATEDTABLE returns a table of related rows and can be used in both contexts.

Information Functions

Function	Description	Use
`ISBLANK(value)`	TRUE if value is blank	Both
`ISNUMBER(value)`	TRUE if value is a number	Both
`ISTEXT(value)`	TRUE if value is text	Both
`ISLOGICAL(value)`	TRUE if value is boolean	Both
`ISERROR(value)`	TRUE if value is an error	Both
`ISEVEN(number)`	TRUE if number is even	Both
`ISODD(number)`	TRUE if number is odd	Both
`CONTAINS(table, col, val)`	TRUE if table contains matching row	Measure
`CONTAINSROW(table, val1, ...)`	TRUE if table contains specific row	Measure
`USERNAME()`	Current user login name	Both
`USERPRINCIPALNAME()`	Current user UPN (email format)	Both
`CUSTOMDATA()`	Value from connection string CustomData	Both

Variables — VAR / RETURN

Variables improve readability and performance by storing intermediate results. They are evaluated once and reused, avoiding repeated calculation.

```
Profit Margin =
VAR TotalRevenue = SUM(Sales[Revenue])
VAR TotalCost    = SUM(Sales[Cost])
VAR Profit       = TotalRevenue - TotalCost
RETURN
    DIVIDE(Profit, TotalRevenue)
```

VAR / RETURN = BOTH (Column & Measure)

Common Patterns

% of Total

```
% of Total Sales =
DIVIDE([Total Sales], CALCULATE([Total Sales], ALL(Sales)))
```

Running Total

```
Running Total =
CALCULATE(
    [Total Sales],
    FILTER(ALL(Date[Date]), Date[Date] <= MAX(Date[Date]))
)
```

Dynamic Ranking

```
Product Rank = RANKX(ALL(Products[Product]), [Total Sales])
```

Prior Period Comparison

```
MoM Change % =
VAR CurrentMonth = [Total Sales]
VAR PriorMonth   = CALCULATE([Total Sales], PREVIOUSMONTH(Date[Date]))
RETURN DIVIDE(CurrentMonth - PriorMonth, PriorMonth)
```

Same Period Last Year

```
SPLY Sales = CALCULATE([Total Sales], SAMEPERIODLASTYEAR(Date[Date]))
```

YTD vs PYTD

```
YTD vs PYTD =
VAR YTD  = TOTALYTD([Total Sales], Date[Date])
VAR PYTD = CALCULATE(TOTALYTD([Total Sales], Date[Date]),
           SAMEPERIODLASTYEAR(Date[Date]))
RETURN DIVIDE(YTD - PYTD, PYTD)
```

Pareto — Cumulative % of Total

```
Cumulative % =
VAR CurrentProduct = SELECTEDVALUE(Products[Product])
VAR CurrentSales   = [Total Sales]
VAR AllProducts    = ALLSELECTED(Products[Product])
VAR BetterProducts = FILTER(AllProducts, CALCULATE([Total Sales]) >= CurrentSales)
RETURN DIVIDE(
    CALCULATE([Total Sales], BetterProducts),
    CALCULATE([Total Sales], AllProducts)
)
```

Column vs Measure — When to Use Which

Use a Calculated Column when...	Use a Measure when...
You need the value per row (e.g. Full Name, Profit per Order)	You need aggregated values (e.g. Total Sales, Average Revenue)

`You need to use it as a slicer or axis`	The value changes based on filter context (slicer, visual)
`You need RELATED to look up from another table`	You are computing ratios, rankings, or time comparisons
`You need to concatenate or label rows`	You want to avoid storing redundant data in the model
`The value is truly fixed per row and does not vary by context`	Performance matters — measures are computed on the fly

Best Practices

- Prefer measures over calculated columns for aggregations — measures respond to filter context, columns do not
- Use VAR / RETURN to make complex measures readable and avoid recalculating the same expression twice
- Use DIVIDE() instead of / — it handles divide-by-zero gracefully with a fallback value
- Always build a dedicated Date table — time intelligence functions require it
- Use ALL() inside CALCULATE to remove unwanted filter context for percentage-of-total patterns
- Avoid FILTER(ALL(...)) when a simpler filter argument works — it is slower
- Name measures clearly: [Total Revenue], [YTD Sales], [MoM Growth %]
- Use SELECTEDVALUE() to safely retrieve a single selected value from a slicer
- Keep calculated columns for row-level attributes — use measures for everything dynamic
- Use USERELATIONSHIP inside CALCULATE to activate inactive relationships for role-playing dimensions